FOREX TRADING SIMULATOR SYSTEM

JOMO KENYATTA UNIVERSITY OF AGRICULTURE AND TECHNOLOGY

NAME: MARVIN RYAN

REGISTRATION NUMBER: ENE211-0147/2023

COURSE: OBJECT ORIENTED PROGRAMMING

YEAR: 2026

Project Documentation Report

Forex Trading Simulator System: Project Documentation

1. Background of the Project

The foreign exchange market (Forex) involves the buying and selling of currencies in order to make profit from changes in market prices. Beginner traders often learn through demo accounts before using real money. This project creates a console-based Forex Trading Simulator using C++ to help students understand how a trading platform can be represented using programming concepts.

The system allows a user to create a demo trading account, select a currency pair, open BUY or SELL trades, calculate profit and loss, update account balance, and store trade history. It is designed to demonstrate basic Object-Oriented Programming concepts such as classes, objects, encapsulation, constructors, functions, file handling, conditional statements, and loops.

2. FUNCTIONAL REQUIREMENTS

Account Management

Create a trading account, view account information, display balance, and update balance after each trade.

Currency Pair Management

Display available trading pairs such as EUR/USD, GBP/USD, and XAU/USD, and provide simulated market prices.

Trade Management

Allow the user to open BUY or SELL trades, enter lot size, and calculate entry and exit prices.

Profit and Loss Calculation

Automatically calculate profit or loss depending on trade type, lot size, entry price, and exit price.

Trade History

Save completed trades in a text file and display previous trading records.

Input Validation

Check user menu choices, trade type, and numerical inputs to reduce errors.

User-Friendly Menu

Provide a simple numbered console menu for easy navigation.

3. SYSTEM ARCHITECTURE

The Forex Trading Simulator uses a simple layered architecture consisting of a user interface layer, an application logic layer, and a data storage layer. The application logic is controlled by the TradingSimulator class, while other classes such as Account, CurrencyPair, and Trade perform specific functions.

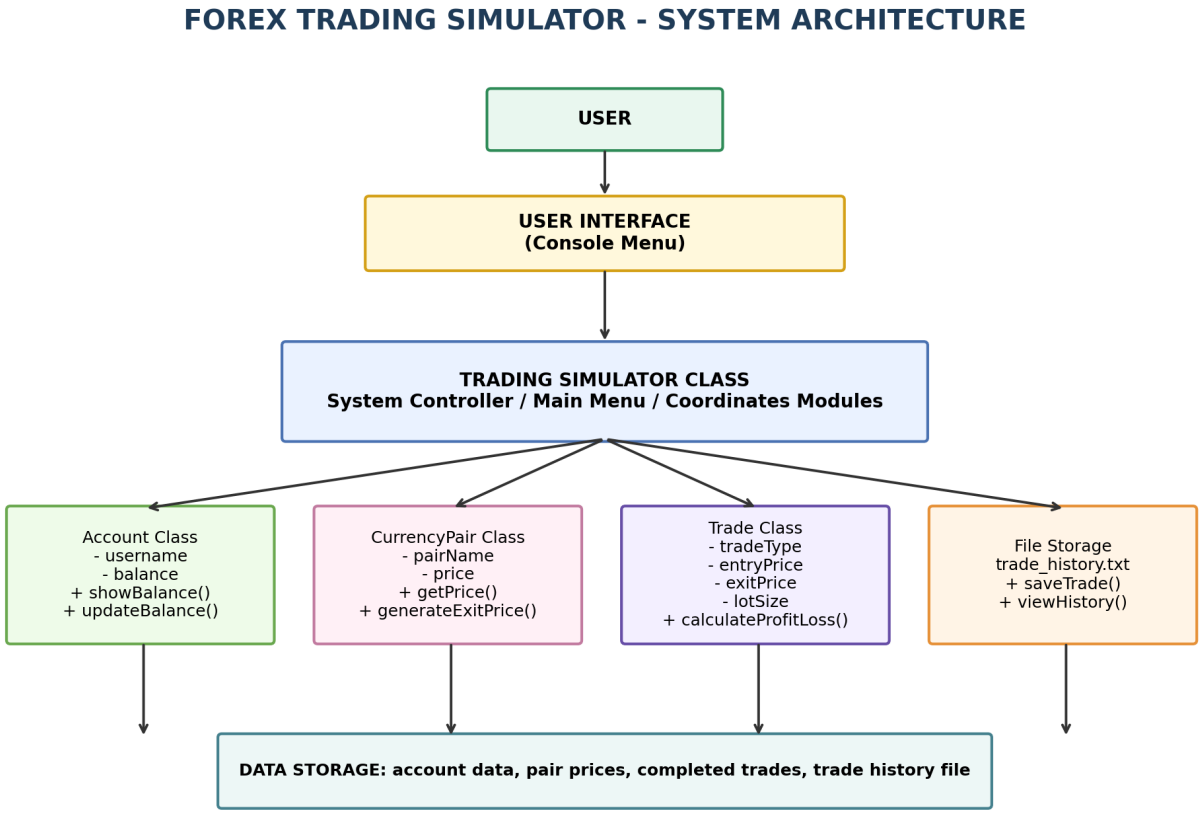

Figure 1: System architecture diagram for the Forex Trading Simulator.

4. MAJOR COMPONENTS

1. Account Management Component

Functions:

- Create account
- Display account details
- Update account balance
- Manage demo funds

Main Class: Account Class

Purpose: Stores username and account balance. This allows the simulator to manage user funds.

2. Currency Pair Management Component

Functions:

- Display currency pairs
- Store pair prices
- Simulate market movement

Main Class: CurrencyPair Class

Purpose: Stores currency pair name, current price, and simulated market movement.

3. Trade Management Component

Functions:

- Open BUY trades
- Open SELL trades
- Enter lot size
- Calculate profit or loss

Main Class: Trade Class

Purpose: Stores trade type, entry price, exit price, lot size, and profit or loss.

4. Trade History Component

Functions:

- Save trades
- Read trade history
- Display previous trades

Main Class: File Storage

Purpose: Uses trade_history.txt to preserve completed trade records.

5. User Interface Component

Functions:

- Display menus
- Accept input
- Show output

Main Class: Console Menu

Purpose: Allows users to interact with the simulator through numbered options.

6. Core System Controller

Functions:

- Controls all modules
- Handles menu navigation
- Coordinates operations

Main Class: TradingSimulator Class

Purpose: Acts as the brain of the entire application.

5. SYSTEM BLOCK DIAGRAM

The system block diagram shows how data flows inside the Forex Trading Simulator. The user interacts with the console menu, and the system controller processes requests by communicating with account management, market simulation, trade execution, profit/loss processing, and file storage modules.

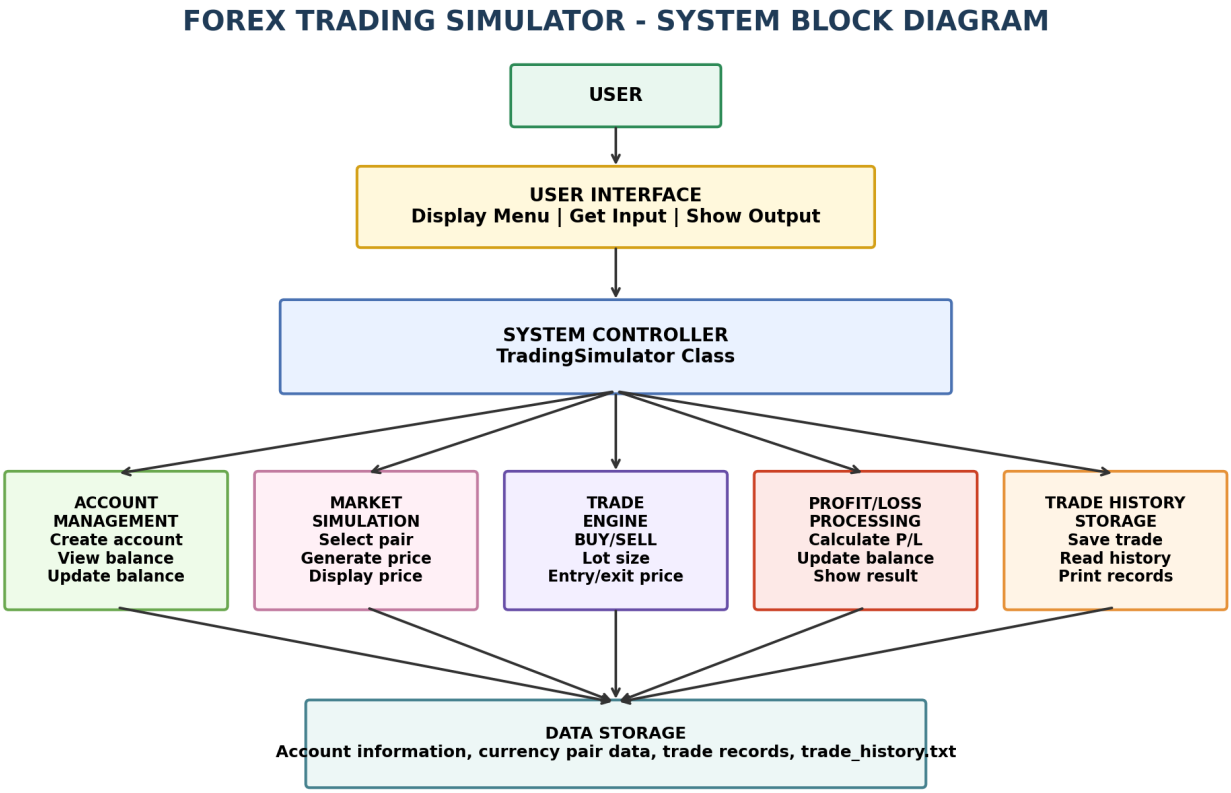

Figure 2: System block diagram showing data flow and system modules.

6. PROGRAM FLOW

- User creates a trading account.
- User selects a currency pair.
- User chooses BUY or SELL.
- User enters lot size.
- System generates simulated market movement.
- Profit or loss is calculated.
- Account balance is updated.
- Trade history is saved.
- User can continue trading or exit the system.

7. OOP CONCEPTS USED

Encapsulation

Data and functions are grouped inside classes. Sensitive data such as balance is declared private and accessed using public methods.

Abstraction

Complex operations are hidden behind simple functions such as calculateProfitLoss(), updateBalance(), and displayTrade().

Classes and Objects

Classes represent real-world trading entities such as Account, CurrencyPair, and Trade. Objects are created from these classes during execution.

Constructors

Constructors initialize objects automatically with user-provided or default values.

Data Hiding

Important variables such as account balance are kept private to protect them from direct access.

Composition

The TradingSimulator class combines and coordinates the Account, CurrencyPair, and Trade classes.

8. SAMPLE OUTPUT

```
===== MAIN MENU =====
1. View Balance
2. Open Trade
3. View Trade History
4. Exit

Pair: XAU/USD
Trade Type: BUY
Entry Price: 2350.00
Exit Price: 2355.20
Lot Size: 1
Profit/Loss: $520
```

9. ADVANTAGES OF THE SYSTEM

- Provides practical understanding of forex trading.
- Demonstrates real-world application of OOP.
- Automatically calculates profit and loss.
- Saves trade history.
- Uses a simple and user-friendly menu system.
- Improves programming and problem-solving skills.
- Demonstrates file handling and data management.

10. LIMITATIONS OF THE SYSTEM

- Uses simulated prices instead of live market prices.
- Console-based interface only.
- Does not use internet connectivity.
- No graphical candlestick charts.
- Limited to basic trading functions.

11. FUTURE IMPROVEMENTS

- Real-time market prices using APIs.
- Graphical User Interface.
- Candlestick chart integration.
- Multiple user accounts.
- Database integration.
- Stop loss and take profit features.

12. CONCLUSION

In conclusion, the Forex Trading Simulator successfully demonstrates the practical application of Object-Oriented Programming concepts in developing a financial trading system. The project allows users to simulate forex trading operations such as account management, currency pair selection, trade execution, and profit/loss calculation through a simple and user-friendly structure.

The modular design improves organization, maintainability, and scalability while helping students understand how real-world trading systems operate. The project effectively combines C++ concepts such as classes, objects, encapsulation, constructors, file handling, and menu-driven interaction into one functional application.

13. REFERENCES

1. Bjarne Stroustrup, The C++ Programming Language.
2. Herbert Schildt, C++: The Complete Reference.
3. Object-Oriented Programming Concepts in C++.
4. Forex Trading Basics - Educational Trading Resources.
5. JKUAT Structured Programming Notes.